

CHAPTER 8 — BINARY SEARCH WITH ARRAYS

Learning Goal

Learn how to implement Binary Search on a **sorted array in C** using **low**, **high**, **mid**, and a target value.

SLIDE 1 — Setting Up Binary Search with an Array

Start with a Sorted Array

Binary Search works with an array whose values are arranged in order.

```
int numbers[] = {10, 20, 30, 40, 50, 60, 70};
```

Array positions:

Value:	10	20	30	40	50	60	70
Index:	0	1	2	3	4	5	6
			↑				
			mid				

 Suppose:

```
int target = 60;
```

Now we need three important positions:

 **low** → first index

 **high** → last index

 **mid** → middle index

```
int low = 0;
```

```
int high = 6;
```

```
int mid = (low + high) / 2;
```

Memory Trick

Array → **Low** → **High** → **Mid** → **Target**

SLIDE 2 — Finding the Middle Element in C

mid Tells Us Where to Look

To calculate the middle:

```
mid = (low + high) / 2;
```

Example:

```
low = 0
```

```
high = 6
```

So:

```
mid = (0 + 6) / 2  
    = 3
```

Therefore:

```
numbers[mid]
```

means:

```
numbers[3] → 40
```

Visual

[10]	[20]	[30]	[40]	[50]	[60]	[70]
0	1	2	3	4	5	6
		↑				
		MID	=	3		

 Target = 60

60 > 40

➡ Target must be in the **right side**.

🧠 **Remember**

mid is an index, not the actual value.

```
mid = 3
numbers[mid] = 40
```

🟡 SLIDE 3 — Updating Low and High

↔ **How Does C Move to the Correct Half?**

After comparing the target with the middle value, we change the search boundaries.

➡ **Target is Greater Than Middle**

```
if(target > numbers[mid])
{
    low = mid + 1;
}
```

We move **low** to the position after **mid**.

⬅ **Target is Smaller Than Middle**

```
if(target < numbers[mid])
{
    high = mid - 1;
}
```

We move **high** to the position before **mid**.

✅ Target Equals Middle

```
if(target == numbers[mid])
{
    printf("Found");
}
```

🎯 Target found!

💡 Memory Trick

Greater → Move LOW ➡

Smaller → Move HIGH ←

Equal → FOUND ✅

🟪 SLIDE 4 — Complete Binary Search Program

💻 Let's Connect Everything!

```
#include <stdio.h>
```

```
int main()
{
    int numbers[] = {10, 20, 30, 40, 50, 60, 70};
    int target = 60;

    int low = 0;
    int high = 6;
    int mid;

    while(low <= high)
    {
        mid = (low + high) / 2;

        if(numbers[mid] == target)
        {
            printf("Found");
            break;
        }
    }
}
```

```

    }
    else if(target < numbers[mid])
    {
        high = mid - 1;
    }
    else
    {
        low = mid + 1;
    }
}

return 0;
}

```

What is each part doing?

 **numbers[]** → stores sorted values

 **target** → value to search

 **low** → beginning of search area

 **high** → end of search area

 **mid** → middle position

 **while** → keeps searching

 **low/high** → reduce the search area

Memory Trick

Set → Find Mid → Compare → Move Boundary → Repeat

SLIDE 5 — Trace the C Program

Let's Watch the Code Work

Array:

10 20 30 40 50 60 70

🎯 Target = 60

Round 1

low = 0

high = 6

mid = 3

numbers[mid] = 40

60 > 40

➡ Update:

low = mid + 1

low = 4

Round 2

low = 4

high = 6

mid = 5

numbers[mid] = 60

60 == 60

✅ FOUND!

🎯 Visual

10 20 30 40 | 50 [60] 70



BIG MEMORY TRICK

● LOW = Start

● HIGH = End

● MID = Check

➡ LOW moves right

⬅ HIGH moves left



CHAPTER 8 — QUIZ



Q1 — Fill the Code

Complete the missing code:

```
int low = 0;  
int high = 6;  
int mid;
```

```
mid = _____;
```



Hint: Calculate the middle using the starting and ending indexes.



Glossary

-  **low** — Starting index of the search area.
-  **high** — Ending index of the search area.
-  **mid** — Middle index of the search area.



Answer:

```
(low + high) / 2
```



Q2 — Fill the Code

Complete the statement:

```
if(target > numbers[mid])  
{  
    _____ = mid + 1;  
}
```



Hint: If the target is greater than the middle value, move the starting boundary to the right.

Glossary

-  **target** — Value we want to find.
-  **mid** — Middle index.
-  **low** — Starting boundary of the search area.

✓ **Answer:** `low`

Q3 — MCQ

Consider:

```
int numbers[] = {10, 20, 30, 40, 50, 60, 70};  
int low = 0;  
int high = 6;
```

```
int mid = (low + high) / 2;
```

What is the value of `mid`?

- A) 2
- B) 3
- C) 4
- D) 6

 **Hint:** Calculate $(0 + 6) / 2$.

Glossary

-  **low** — Starting index.
-  **high** — Ending index.
-  **mid** — Middle index.

✓ **Answer:** B) 3

Q4 — MCQ

What should happen when:

`target < numbers[mid]`

in a sorted ascending array?

- A) Move **low** to the right
- B) Move **high** to the left
- C) Stop the program immediately
- D) Sort the array again

 **Hint:** If the target is smaller than the middle value, which side should we search?

Glossary

-  **target** — Value being searched for.
-  **mid** — Middle position.
-  **high** — Ending boundary of the search area.

 **Answer: B) Move **high** to the left**

Q5 — Fill in the Blank

In Binary Search, when:

`target > numbers[mid]`

the search continues in the _____ **side** of the sorted array.

 **Hint:** Larger values are on which side of an ascending array?

Glossary

-  **target** — Value we want to find.
-  **mid** — Middle position.
-  **side** — Direction in which the search continues.

 **Answer: right**

Q6 — Fill in the Blank

When:

`target == numbers[mid]`

the target is _____.

 **Hint:** The target and middle value are equal.

Glossary

-  **target** — Value we are searching for.
-  **mid** — Middle position of the search area.
-  **equal** — Having the same value.

 **Answer: found**